

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA CƠ BẢN I

THỰC TẬP CƠ SỞ



BÁO CÁO TUẦN

Đề tài:

Xây dựng website tuyển dụng việc làm

Giảng viên hướng dẫn	: Kim Ngọc Bách
Họ và tên sinh viên	: Đỗ Bình Minh
Ngày sinh	: 24/12/2005
Số điện thoại	: 0329279863
Mã sinh viên	: B23DCCN542
Lớp	: D23CQCN10-B
Nhóm lớp	: 11

Hà Nội – 2026

Mục lục

1. Tóm tắt các công việc đã thực hiện trong tuần	3
2. Chi các công việc thực hiện trong tuần	3
2.1. Tính năng phân trang.....	3
2.1.1. Xử lý bên phía Frontend.....	3
2.1.2. Xử lý bên phía backend.....	5
2.1.3. Kết quả	6
2.2. Trang chi tiết công việc	8
2.2.1. Xử lý bên phía Frontend.....	8
2.2.2. Xử lý bên phía Backend	9
2.2.3. Kết quả.	12
2.3. Form ứng tuyển	13
2.3.1 Xử lý bên phía Frontend.....	13
2.3.2. Xử lý bên phía Backend	15
2.3.3. Kết quả	20
3. Kết quả đạt được	21
4. Khó khăn gặp phải.....	21
5. Hướng xử lý.....	22
6. Kế hoạch tuần tiếp theo	22

1. Tóm tắt các công việc đã thực hiện trong tuần

Trong tuần này, em tiếp tục hoàn thiện website tuyển dụng việc làm với các chức năng chính gồm: phân trang danh sách công việc, xây dựng trang chi tiết công việc và xây dựng form ứng tuyển.

Ở chức năng phân trang, em đã xử lý phía frontend để lưu trang hiện tại, tổng số trang và tổng số bản ghi. Khi người dùng thay đổi trang, hệ thống sẽ gọi lại API tìm kiếm để lấy danh sách công việc tương ứng. Ở phía backend, em bổ sung xử lý phân trang bằng cách giới hạn số lượng công việc mỗi trang, tính số bản ghi cần bỏ qua và trả về tổng số trang, tổng số bản ghi cho frontend.

Bên cạnh đó, em đã xây dựng trang chi tiết công việc. Frontend lấy id công việc từ URL, gọi API để lấy dữ liệu chi tiết và hiển thị thông tin công việc. Backend xử lý việc tìm công việc theo id, lấy thêm thông tin công ty liên quan và trả về dữ liệu chi tiết cho giao diện.

Ngoài ra, em cũng thực hiện chức năng form ứng tuyển. Frontend kiểm tra dữ liệu nhập vào như họ tên, email, số điện thoại và file CV. Sau khi dữ liệu hợp lệ, hệ thống gửi thông tin ứng tuyển lên backend. Backend xử lý upload file CV, kiểm tra dữ liệu, lưu thông tin ứng tuyển vào MongoDB thông qua model CV.

2. Chi các công việc thực hiện trong tuần

2.1. Tính năng phân trang

2.1.1. Xử lý bên phía Frontend

```
import { useEffect, useState } from "react";

export const SearchContainer = () => {
  const [page, setPage] = useState(1);
  const [totalPage, setTotalPage] = useState(0);
  const [totalRecord, setTotalRecord] = useState(0);
  const searchParams = useSearchParams();
  const router = useRouter();
  const language = searchParams.get("language") || "";
  @@ -17,16 +20,18 @@ export const SearchContainer = () => {
  const [jobList, setJobList] = useState<any[]>([]);

  useEffect(() => {
    fetch(`${process.env.NEXT_PUBLIC_API_URL}/search?
    language=${language}&city=${city}&company=${company}&keyword=${keyword}&position=${position}&workingForm=${
    workingForm}&page=${page}`, {
      method: "GET"
    })
    .then(res => res.json())
    .then(data => {
      if(data.code == "success") {
        setJobList(data.jobs);
        setTotalPage(data.totalPage);
        setTotalRecord(data.totalRecord);
      }
    })
  }, [language, city, company, keyword, position, workingForm, page]);

  const handleFilterStatus = (event: any) => {
    const value = event.target.value;
    @@ -50,10 +55,15 @@ export const SearchContainer = () => {
    router.push(`?${params.toString()}`);
  }

  const handlePagination = (event: any) => {
    const value = event.target.value;
    setPage(parseInt(value));
  }
}
```

Đoạn code trên xây dựng component SearchContainer dùng để tìm kiếm và phân trang danh sách việc làm. Component sử dụng useState để lưu trang hiện tại (page), tổng số trang (totalPage) và tổng số bản ghi (totalRecord). Các điều kiện tìm kiếm như ngôn ngữ, thành phố, công ty, từ khóa, vị trí và hình thức làm việc được lấy từ URL bằng useSearchParams.

Khi người dùng thay đổi bộ lọc hoặc chuyển trang, useEffect sẽ tự động gọi API /search bằng phương thức GET. API nhận các tham số tìm kiếm và số trang hiện tại, sau đó trả về danh sách công việc, tổng số trang và tổng số bản ghi. Dữ liệu trả về được lưu vào các state như jobList, totalPage, totalRecord để hiển thị lên giao diện.

Hàm handleFilterStatus dùng để xử lý khi người dùng thay đổi bộ lọc, đồng thời đưa trang hiện tại về trang 1 và cập nhật lại URL. Hàm handlePagination dùng để xử lý khi người dùng chọn trang khác, sau đó cập nhật lại state page, làm cho danh sách việc làm được tải lại theo trang mới.

```
{totalRecord} việc làm <span className="text-[#0088FF]">{language} {city} {company} {keyword}</span>

<select
  name=""
  className="border border-[#DEDEDE] rounded-[8px] py-[12px] px-[18px] font-[400] text-[16px] text-[#414042]"
  onChange={handlePagination}
>
  {Array(totalPage).fill("").map((item, index) => (
    <option key={index} value={index+1}>
      Trang {index+1}
    </option>
  ))}
</select>
```

Đoạn code này dùng để hiển thị tổng số việc làm tìm được sau khi tìm kiếm. Biến totalRecord thể hiện tổng số bản ghi được trả về từ API. Các giá trị như language, city, company, keyword được hiển thị kèm theo để người dùng biết kết quả đang được lọc theo điều kiện nào.

Phần thẻ select dùng để tạo chức năng chọn trang trong phân trang. Dựa vào biến totalPage, chương trình dùng Array(totalPage).fill("").map() để tạo ra danh sách các trang từ 1 đến tổng số trang. Mỗi option tương ứng với một trang, ví dụ: Trang 1, Trang 2, Trang 3,...

Khi người dùng chọn một trang khác, sự kiện onChange sẽ gọi hàm handlePagination. Hàm này lấy giá trị trang được chọn và cập nhật lại state page, từ đó làm cho API được gọi lại để lấy danh sách việc làm của trang tương ứng.

2.1.2. Xử lý bên phía backend

```
export const search = async (req: Request, res: Response) => {
  const dataFinal = [];
  let totalPage = 0;
  let totalRecord = 0;

  if(Object.keys(req.query).length > 0) {
    const find: any = {};
    find.workingForm = req.query.workingForm;
  }

  // Phân trang
  const limitItems = 2;
  let page = 1;
  if(req.query.page && parseInt(`${req.query.page}`) > 0) {
    page = parseInt(`${req.query.page}`);
  }
  const skip = (page - 1) * limitItems;
  totalRecord = await Job.countDocuments(find);
  totalPage = Math.ceil(totalRecord/limitItems);
  // Hết Phân trang

  const jobs = await Job
    .find(find)
    .sort({
      createdAt: "desc"
    })
    .limit(limitItems)
    .skip(skip);

  for (const item of jobs) {
    const itemFinal = {
      code: "success",
      message: "Thành công!",
      jobs: dataFinal,
      totalPage: totalPage,
      totalRecord: totalRecord
    };
  }
}
```

Đoạn code này là API search ở phía backend, dùng để tìm kiếm và phân trang danh sách việc làm. Ban đầu, hệ thống tạo các biến dataFinal, totalPage, totalRecord để lưu dữ liệu trả về, tổng số trang và tổng số bản ghi.

Nếu request có dữ liệu tìm kiếm trong req.query, hệ thống sẽ lấy các điều kiện như city, workingForm,... và đưa vào biến find để lọc dữ liệu trong database.

Phân trang được xử lý bằng biến limitItems = 2, nghĩa là mỗi trang hiển thị 2 việc làm. Biến page lấy từ req.query.page, nếu không có thì mặc định là trang 1. Sau đó hệ thống tính:

```
const skip = (page - 1) * limitItems;
```

Biến skip dùng để bỏ qua số bản ghi của các trang trước. Tiếp theo, backend dùng `countDocuments(find)` để đếm tổng số việc làm phù hợp điều kiện tìm kiếm, rồi tính tổng số trang bằng:

```
totalPage = Math.ceil(totalRecord / limitItems);
```

Cuối cùng, hệ thống truy vấn danh sách việc làm bằng `find(find)`, sắp xếp theo `createdAt` giảm dần, giới hạn số lượng bằng `limit(limitItems)` và bỏ qua dữ liệu bằng `skip(skip)`. Sau khi xử lý xong, API trả về JSON gồm danh sách việc làm, tổng số trang và tổng số bản ghi.

2.1.3. Kết quả

The screenshot displays a web interface for job listings. At the top, there's a header with the text "5 việc làm Đà Nẵng". Below this, there are two filter buttons: "Cấp bậc" (Level) and "Hình thức làm việc" (Work form). The main content area shows two job cards side-by-side. Each card features a Facebook logo, a job title, company name, salary range, experience level, work type, location, and a list of technologies. The first card is for "Công việc 5" (Job 5) and the second is for "Công việc 4" (Job 4).

Job Title	Company	Salary Range	Experience Level	Work Type	Location	Technologies
Công việc 5	ABC.LTD	900\$ - 1.200\$	Junior	Làm từ xa	Đà Nẵng	HTML5, CSS3, Javascript, ReactJS
Công việc 4	ABC.LTD	600\$ - 900\$	Fresher	Tại văn phòng	Đà Nẵng	HTML5, CSS3, Javascript, ReactJS



Công việc 5
ABC.LTD
900\$ - 1.200\$
👤 Junior
💼 Làm từ xa
📍 Đà Nẵng

HTML5

CSS3

Javascript

ReactJS

NodeJS

PHP



Công việc 4
ABC.LTD
600\$ - 900\$
👤 Fresher
💼 Tại văn phòng
📍 Đà Nẵng

HTML5

CSS3

Javascript

ReactJS

NodeJS

PHP

Trang 1 ▾

Số công việc phù hợp với kết quả tìm kiếm sẽ được hiển thị đồng thời được chia ra thành từng trang để giao diện gọn hơn.

2.2. Trang chi tiết công việc

2.2.1. Xử lý bên phía Frontend

```
export default async function JobDetailPage({ params }: {
  params: {
    slug: string
  }
}) {
  const { slug } = await params;

  const res = await fetch(`${process.env.NEXT_PUBLIC_API_URL}/job/detail/${slug}`);
  const data = await res.json();
  let jobDetail: any = null;
  let position: any = null;
  let workingForm: any = null;
  if(data.code == "success") {
    jobDetail = data.jobDetail;
    position = positionList.find(pos => pos.value == jobDetail.position);
    workingForm = workingFormList.find(work => work.value == jobDetail.workingForm);
  }
}
```

Đoạn code này xây dựng trang chi tiết việc làm ở frontend bằng Next.js. Hàm JobDetailPage nhận tham số params, trong đó slug chính là id của công việc được truyền trên URL.

```
const { slug } = await params;
```

Dòng này lấy id công việc từ đường dẫn.

Sau đó, frontend gọi API đến backend để lấy thông tin chi tiết việc làm:

```
const res = await
fetch(`${process.env.NEXT_PUBLIC_API_URL}/job/detail/${slug}`);
const data = await res.json();
```

API sẽ trả về dữ liệu chi tiết của công việc tương ứng với slug.

Các biến jobDetail, position, workingForm được khai báo ban đầu là null. Nếu API trả về thành công, chương trình sẽ gán dữ liệu chi tiết việc làm vào jobDetail.

```
if(data.code == "success") {
  jobDetail = data.jobDetail;
}
```

Sau đó, chương trình tìm thông tin hiển thị tương ứng cho vị trí công việc và hình thức làm việc từ các danh sách có sẵn:

```
position = positionList.find(pos => pos.value == jobDetail.position);
workingForm = workingFormList.find(work => work.value == jobDetail.workingFo
```

2.2.2. Xử lý bên phía Backend

```
import { Router } from "express";
import * as jobController from "../controllers/job.controller";

const router = Router();

router.get("/detail/:id", jobController.detail)

export default router;
```

Đoạn code này dùng để khai báo route cho chức năng xem chi tiết việc làm ở backend.

```
import jobRoutes from "./job.route";

router.use("/job", jobRoutes)
```

Đoạn code này dùng để gắn các route liên quan đến việc làm vào hệ thống route chính của backend.

```

import { Request, Response } from "express";
import Job from "../models/job.model";
import AccountCompany from "../models/account-company.model";

export const detail = async (req: Request, res: Response) => {
  try {
    const id = req.params.id;

    const record = await Job.findOne({
      _id: id
    })

    if(!record) {
      res.json({
        code: "error",
        message: "Thất bại!"
      })
      return;
    }

    const companyInfo = await AccountCompany.findOne({
      _id: record.companyId
    })

    if(!companyInfo) {
      res.json({
        code: "error",
        message: "Thất bại!"
      })
      return;
    }

    const jobDetail = {
      id: record.id,
      title: record.title,
      companyName: companyInfo.companyName,
      salaryMin: record.salaryMin,
      salaryMax: record.salaryMax,
      images: record.images,
      position: record.position,
      workingForm: record.workingForm,
      companyAddress: companyInfo.address,
      technologies: record.technologies,
      description: record.description,
      companyLogo: companyInfo.logo,
      companyId: record.companyId,
      companyModel: companyInfo.companyModel,
      companyEmployees: companyInfo.companyEmployees,
      workingTime: companyInfo.workingTime,
      workOvertime: companyInfo.workOvertime
    };

    res.json({
      code: "success",
      message: "Thành công!",
      jobDetail: jobDetail
    })
  } catch (error) {
    res.json({
      code: "error",
      message: "Thất bại!"
    })
  }
}

```

Đoạn code này xây dựng hàm detail ở backend, dùng để lấy thông tin chi tiết của một việc làm theo id.

Đầu tiên, hệ thống lấy id từ đường dẫn URL:

```
const id = req.params.id;
```

Sau đó, chương trình tìm việc làm trong bảng Job theo `_id`:

```
const record = await Job.findOne({ _id: id });
```

Nếu không tìm thấy việc làm, API sẽ trả về kết quả thất bại.

Tiếp theo, hệ thống dùng `companyId` của việc làm để tìm thông tin công ty trong bảng `AccountCompany`:

```
const companyInfo = await AccountCompany.findOne({  
  _id: record.companyId  
});
```

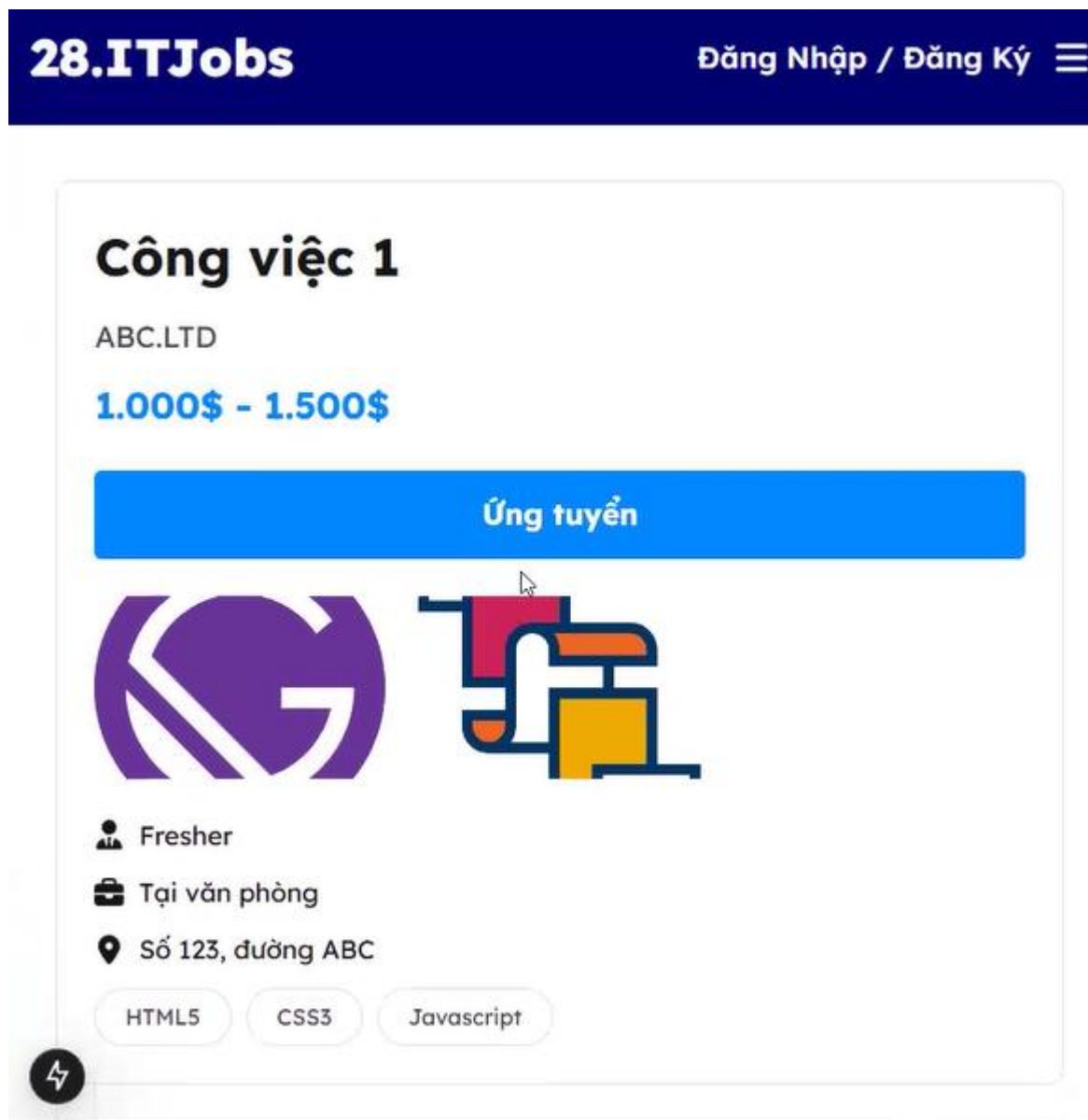
Nếu không tìm thấy thông tin công ty, API cũng trả về lỗi.

Sau khi lấy được cả thông tin việc làm và thông tin công ty, chương trình tạo đối tượng `jobDetail` để gom các dữ liệu cần hiển thị như: tên công việc, tên công ty, mức lương, hình ảnh, vị trí, hình thức làm việc, công nghệ, mô tả, logo công ty, địa chỉ, quy mô công ty và thời gian làm việc.

Cuối cùng, API trả về dữ liệu chi tiết việc làm cho frontend:

```
res.json({  
  code: "success",  
  message: "Thành công!",  
  jobDetail: jobDetail  
});
```

2.2.3. Kết quả.



Khi chọn 1 công việc chi tiết công việc sẽ hiển thị cho người dùng

2.3. Form ứng tuyển

2.3.1 Xử lý bên phía Frontend

```
useEffect(() => {
  const validator = new JustValidate("#applyForm");

  validator
    .addField('#fullName', [
      {
        rule: 'required',
        errorMessage: 'Vui lòng nhập họ tên!'
      },
      {
        rule: 'minLength',
        value: 5,
        errorMessage: 'Họ tên phải có ít nhất 5 ký tự!',
      },
      {
        rule: 'maxLength',
        value: 50,
        errorMessage: 'Họ tên không được vượt quá 50 ký tự!',
      },
    ])
    .addField('#email', [
      {
        rule: 'required',
        errorMessage: 'Vui lòng nhập email của bạn!',
      },
      {
        rule: 'email',
        errorMessage: 'Email không đúng định dạng!',
      },
    ])
    .addField('#phone', [
      {
        rule: 'required',
        errorMessage: 'Vui lòng nhập số điện thoại!'
      },
      {
        rule: 'customRegexp',
        value: /^(04|03|5|7|8|9|)+([0-9]{8})\b/g,
        errorMessage: 'Số điện thoại không đúng định dạng!'
      },
    ])
    .addField('#fileCV', [
      {
        rule: 'required',
        errorMessage: 'Vui lòng chọn file CV!',
      },
      {
        validator: (value: any, fields: any) => {
          const file = fields['#fileCV']?.elem?.files?.[0];
          if (!file) return false;
          return file.type === 'application/pdf';
        },
        errorMessage: 'File phải là định dạng PDF!',
      },
      {
        validator: (value: any, fields: any) => {
          const file = fields['#fileCV']?.elem?.files?.[0];
          if (!file) return false;
          return file.size <= 5 * 1024 * 1024; // 5MB = 5 * 1024 KB * 1024 bytes
        },
        errorMessage: 'Dung lượng file không được vượt quá 5MB!',
      },
    ])
  })
```

Đoạn code này dùng để kiểm tra dữ liệu trong form ứng tuyển việc làm bằng thư viện JustValidate.

Khi component được render, useEffect sẽ khởi tạo đối tượng validate cho form có id là applyForm:

```
const validator = new JustValidate("#applyForm");
```

Sau đó, chương trình thêm các điều kiện kiểm tra cho từng ô nhập liệu.

Với trường fullName, hệ thống yêu cầu người dùng phải nhập họ tên, họ tên phải có ít nhất 5 ký tự và không được vượt quá 50 ký tự.

Với trường email, hệ thống kiểm tra người dùng đã nhập email chưa và email có đúng định dạng hay không.

Với trường phone, hệ thống yêu cầu nhập số điện thoại và kiểm tra số điện thoại đúng định dạng bằng biểu thức chính quy regex.

Với trường fileCV, hệ thống kiểm tra người dùng đã chọn file CV chưa. Sau đó kiểm tra file có đúng định dạng PDF hay không và dung lượng file không được vượt quá 5MB.

Nếu người dùng nhập sai hoặc thiếu dữ liệu, hệ thống sẽ hiển thị thông báo lỗi tương ứng như “Vui lòng nhập họ tên”, “Email không đúng định dạng”, “File phải là định dạng PDF”,...

```
.onSuccess((event: any) => {
  const fullName = event.target.fullName.value;
  const email = event.target.email.value;
  const phone = event.target.phone.value;
  const fileCV = event.target.fileCV.files[0];

  // Tạo FormData
  const formData = new FormData();
  formData.append("jobId", jobId);
  formData.append("fullName", fullName);
  formData.append("email", email);
  formData.append("phone", phone);
  formData.append("fileCV", fileCV);

  fetch(`${process.env.NEXT_PUBLIC_API_URL}/job/apply`, {
    method: "POST",
    body: formData
  })
    .then(res => res.json())
    .then(data => {
      if(data.code === "error") {
        toast.error(data.message);
      }

      if(data.code === "success") {
        toast.success(data.message);
        event.target.reset();
      }
    })
  })
}, []);
```

Đoạn code này xử lý sự kiện khi người dùng gửi form ứng tuyển thành công sau khi đã vượt qua bước kiểm tra dữ liệu.

Trong hàm onSuccess, hệ thống lấy dữ liệu từ form gồm họ tên, email, số điện thoại và file CV:

```
const fullName = event.target.fullName.value;
const email = event.target.email.value;
const phone = event.target.phone.value;
const fileCV = event.target.fileCV.files[0];
```

Sau đó, chương trình tạo đối tượng FormData để gửi dữ liệu lên backend, đặc biệt phù hợp khi form có chứa file upload như CV.

```
const formData = new FormData();
formData.append("jobId", jobId);
formData.append("fullName", fullName);
formData.append("email", email);
formData.append("phone", phone);
formData.append("fileCV", fileCV);
```

Tiếp theo, frontend gửi request POST đến API /job/apply để nộp hồ sơ ứng tuyển:

```
fetch(`${process.env.NEXT_PUBLIC_API_URL}/job/apply`, {
  method: "POST",
  body: formData
})
```

Sau khi backend trả về kết quả, nếu thất bại thì hiển thị thông báo lỗi bằng toast.error. Nếu thành công thì hiển thị thông báo thành công bằng toast.success và reset lại form.

2.3.2. Xử lý bên phía Backend

```
import { Router } from "express";
import * as jobController from "../controllers/job.controller";
import multer from "multer";
import { storage } from "../helpers/cloudinary.helper";
import * as jobValidate from "../validates/job.validate";

const router = Router();

const upload = multer({ storage: storage });

router.get("/detail/:id", jobController.detail)

router.post(
  "/apply",
  upload.single("fileCV"),
  jobValidate.applyPost,
  jobController.applyPost
)
```

Đoạn code này khai báo các route liên quan đến việc làm ở backend, bao gồm chức năng xem chi tiết việc làm và nộp hồ sơ ứng tuyển.

Đầu tiên, hệ thống import các thư viện và file cần thiết:

```
import { Router } from "express";
import * as jobController from "../controllers/job.controller";
import multer from "multer";
import { storage } from "../helpers/cloudinary.helper";
import * as jobValidate from "../validates/job.validate";
```

Trong đó:

Router dùng để tạo route trong Express.

jobController chứa các hàm xử lý logic như xem chi tiết việc làm và ứng tuyển.

multer dùng để xử lý upload file.

storage là cấu hình lưu file lên Cloudinary.

jobValidate dùng để kiểm tra dữ liệu trước khi xử lý ứng tuyển.

Dòng sau tạo cấu hình upload file:

```
const upload = multer({ storage: storage });
```

Nghĩa là khi người dùng upload CV, file sẽ được xử lý bằng multer và lưu theo cấu hình storage.

Route này dùng để xem chi tiết việc làm:

```
router.get("/detail/:id", jobController.detail);
```

Khi client gọi API /job/detail/:id, hệ thống sẽ lấy id công việc và gọi hàm detail để trả về thông tin chi tiết.

Route này dùng để ứng tuyển việc làm:

```
router.post(
  "/apply",
  upload.single("fileCV"),
  jobValidate.applyPost,
  jobController.applyPost
);
```

Khi người dùng gửi form ứng tuyển, hệ thống sẽ nhận request POST /job/apply. Trước tiên, upload.single("fileCV") xử lý file CV được gửi lên. Sau đó, jobValidate.applyPost

kiểm tra dữ liệu form. Nếu dữ liệu hợp lệ, jobController.applyPost sẽ lưu thông tin ứng tuyển vào hệ thống.

```
import { Request, Response } from "express";
import Job from "../models/job.model";
import AccountCompany from "../models/account-company.model";
import CV from "../models/cv.model";

export const detail = async (req: Request, res: Response) => {
  try {
    @@ -62,3 +63,23 @@ export const detail = async (req: Request, res: Response) => {
  })
}
}

export const applyPost = async (req: Request, res: Response) => {
  try {
    req.body.fileCV = req.file ? req.file.path : "";

    const newRecord = new CV(req.body);
    await newRecord.save();

    res.json({
      code: "success",
      message: "Đã gửi CV thành công!"
    });
  } catch (error) {
    console.log(error);
    res.json({
      code: "success",
      message: "Dữ liệu không hợp lệ!"
    });
  }
}
```

Đoạn code này xử lý chức năng nộp CV ứng tuyển việc làm ở backend.

Hàm applyPost nhận dữ liệu từ form ứng tuyển do frontend gửi lên. Dữ liệu gồm các thông tin như họ tên, email, số điện thoại, id công việc và file CV.

```
req.body.fileCV = req.file ? req.file.path : "";
```

Dòng này kiểm tra xem người dùng có upload file CV hay không. Nếu có, hệ thống lấy đường dẫn file từ req.file.path và gán vào req.body.fileCV. Nếu không có file thì gán giá trị rỗng.

Sau đó, hệ thống tạo một bản ghi CV mới:

```
const newRecord = new CV(req.body);
await newRecord.save();
```

Dữ liệu ứng tuyển được lưu vào collection cvs trong MongoDB thông qua model CV.

Nếu lưu thành công, backend trả về kết quả:

```
res.json({
  code: "success",
```

```
message: "Đã gửi CV thành công!"
});
```

Nếu có lỗi trong quá trình xử lý, hệ thống bắt lỗi bằng catch và trả về thông báo dữ liệu không hợp lệ.

```
import { Request, Response, NextFunction } from "express";
import Joi from "joi";

export const applyPost = (req: Request, res: Response, next: NextFunction) => {
  const schema = Joi.object({
    jobId: Joi.string()
      .required()
      .messages({
        "string.empty": "Không tìm thấy công việc này!"
      }),
    fullName: Joi.string()
      .required()
      .min(5)
      .max(50)
      .messages({
        "string.empty": "Vui lòng nhập họ tên!",
        "string.min": "Họ tên phải có ít nhất 5 ký tự!",
        "string.max": "Họ tên không được vượt quá 50 ký tự!"
      }),
    email: Joi.string()
      .required()
      .email()
      .messages({
        "string.empty": "Vui lòng nhập email!",
        "string.email": "Email không đúng định dạng!"
      }),
    phone: Joi.string()
      .required()
      .custom((value, helpers) => {
        if (!/(84|0[3|5|7|8|9])+([0-9]{8})\b/g.test(value)) {
          return helpers.error("phone.invalid"); // Định dạng số điện thoại Việt Nam
        }
        return value;
      })
      .messages({
        "string.empty": "Vui lòng nhập số điện thoại!",
        "phone.invalid": "Số điện thoại không đúng định dạng!"
      }),
    fileCV: Joi.string().allow('')
  });

  const { error } = schema.validate(req.body);

  if(error) {
    const errorMessage = error.details[0].message;

    res.json({
      code: "error",
      message: errorMessage
    });
    return;
  }

  next();
}
```

Đoạn code này dùng để kiểm tra dữ liệu đầu vào khi người dùng nộp CV ứng tuyển việc làm ở backend. Middleware applyPost sử dụng thư viện Joi để tạo schema validate cho các trường dữ liệu được gửi từ frontend.

Các trường được kiểm tra gồm:

jobId: bắt buộc phải có, dùng để xác định công việc mà ứng viên ứng tuyển.

fullName: bắt buộc nhập, độ dài tối thiểu 5 ký tự và tối đa 50 ký tự.

email: bắt buộc nhập và phải đúng định dạng email.

phone: bắt buộc nhập và được kiểm tra bằng regex để đảm bảo đúng định dạng số điện thoại Việt Nam.

fileCV: cho phép giá trị rỗng, vì file CV đã được xử lý riêng thông qua middleware upload file.

Sau khi tạo schema, hệ thống kiểm tra dữ liệu bằng:

```
const { error } = schema.validate(req.body);
```

Nếu dữ liệu không hợp lệ, backend lấy thông báo lỗi đầu tiên và trả về cho frontend:

```
res.json({  
  code: "error",  
  message: errorMessage  
});
```

Nếu dữ liệu hợp lệ, hàm next() sẽ được gọi để chuyển request sang controller xử lý lưu CV vào database

```
import mongoose from "mongoose";  
  
const schema = new mongoose.Schema(  
  {  
    jobId: String,  
    fullName: String,  
    email: String,  
    phone: String,  
    fileCV: String,  
    viewed: {  
      type: Boolean,  
      default: false  
    },  
    status: {  
      type: String,  
      default: "initial"  
    }  
  },  
  {  
    timestamps: true, // Tự động sinh ra trường createdAt và updatedAt  
  }  
);  
  
const CV = mongoose.model('CV', schema, "cvs");  
  
export default CV;
```

Đoạn code này định nghĩa model CV bằng thư viện mongoose, dùng để lưu thông tin hồ sơ ứng tuyển vào MongoDB.

Schema gồm các trường chính như:

jobId: lưu id của công việc mà ứng viên ứng tuyển.

fullName: lưu họ tên ứng viên.

email: lưu email ứng viên.

phone: lưu số điện thoại ứng viên.

fileCV: lưu đường dẫn hoặc URL file CV đã upload.

viewed: dùng để đánh dấu CV đã được nhà tuyển dụng xem hay chưa, kiểu dữ liệu là Boolean, mặc định là false.

status: lưu trạng thái xử lý hồ sơ, kiểu dữ liệu là String, mặc định là "initial".

Phần cấu hình:

timestamps: true

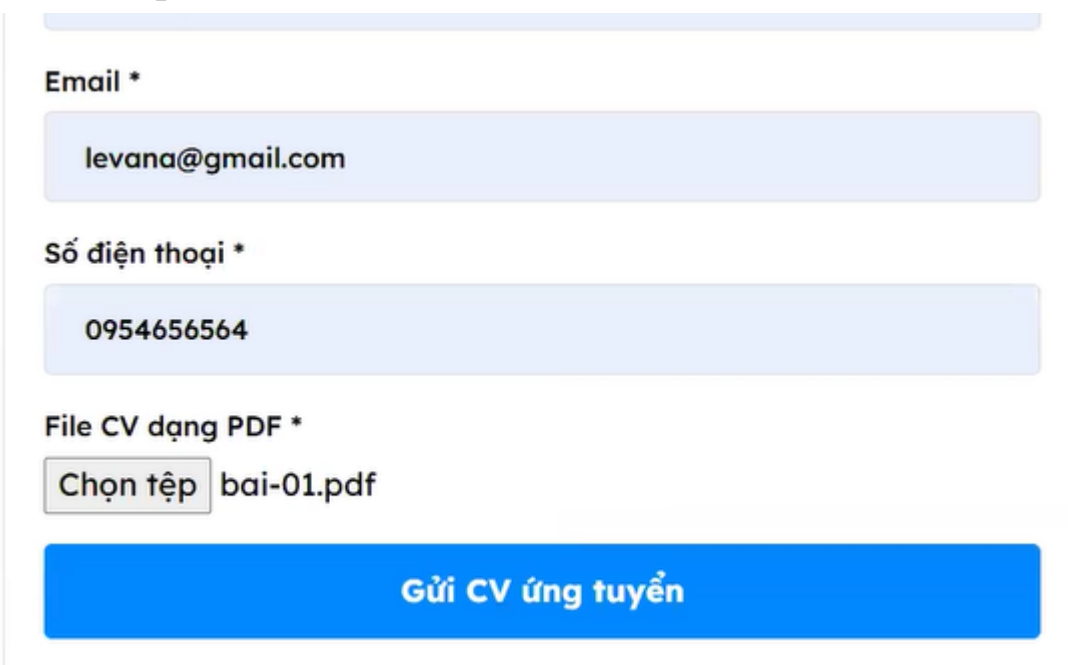
giúp MongoDB tự động sinh thêm hai trường createdAt và updatedAt, dùng để lưu thời gian tạo và cập nhật hồ sơ.

Cuối cùng, đoạn code tạo model:

```
const CV = mongoose.model("CV", schema, "cvs");
```

Model này liên kết với collection cvs trong MongoDB và được export để sử dụng trong các controller khác.

2.3.3. Kết quả



The screenshot shows a web form for job application. It contains three input fields: 'Email *' with the value 'levana@gmail.com', 'Số điện thoại *' with the value '0954656564', and 'File CV dạng PDF *' with a file selection button labeled 'Chọn tệp' and the filename 'bai-01.pdf'. At the bottom is a large blue button labeled 'Gửi CV ứng tuyển'.

```
_id: ObjectId('690b40f39981bb46d5733cd7')
jobId: "68ff5f736c4a2020784dc2b7"
fullName: "Le Van A"
email: "levana@gmail.com"
phone: "0954656564"
fileCV: "https://res.cloudinary.com/dxhwpfxxn/image/upload/v1762345198/putghkbr..."
viewed: false
status: "initial"
createdAt: 2025-11-05T12:20:03.050+00:00
updatedAt: 2025-11-05T12:20:03.050+00:00
__v: 0
```

Khi gửi form ứng tuyển thành công dữ liệu sẽ được lưu vào cơ sở dữ liệu online

3. Kết quả đạt được

Trong tuần này, em đã hoàn thiện được chức năng phân trang cho trang tìm kiếm việc làm. Danh sách công việc sau khi tìm kiếm được chia thành từng trang, giúp giao diện gọn gàng hơn và người dùng dễ theo dõi kết quả hơn.

Em cũng đã xây dựng được trang chi tiết công việc. Khi người dùng chọn một công việc trong danh sách, hệ thống sẽ chuyển sang trang chi tiết và hiển thị đầy đủ các thông tin như tên công việc, tên công ty, mức lương, vị trí, hình thức làm việc, công nghệ, mô tả công việc, địa chỉ công ty và các thông tin liên quan khác.

Ngoài ra, em đã hoàn thiện chức năng ứng tuyển công việc. Người dùng có thể nhập thông tin cá nhân, tải lên file CV và gửi hồ sơ ứng tuyển. Dữ liệu sau khi gửi thành công được lưu vào cơ sở dữ liệu online, đồng thời hệ thống hiển thị thông báo kết quả cho người dùng.

4. Khó khăn gặp phải

Trong quá trình thực hiện, em gặp khó khăn khi kết hợp chức năng tìm kiếm, lọc và phân trang trên cùng một API. Khi người dùng thay đổi bộ lọc hoặc chuyển trang, frontend cần gọi lại API đúng với các tham số hiện tại để dữ liệu hiển thị chính xác.

Ở chức năng chi tiết công việc, dữ liệu cần hiển thị không chỉ nằm trong bảng công việc mà còn liên quan đến bảng công ty. Vì vậy, backend phải truy vấn thêm thông tin công ty dựa vào companyId của công việc để trả về dữ liệu đầy đủ cho frontend.

Đối với form ứng tuyển, em gặp khó khăn trong việc xử lý upload file CV và kiểm tra dữ liệu đầu vào. File CV cần được kiểm tra đúng định dạng PDF, dung lượng không vượt quá giới hạn, đồng thời backend cũng cần validate lại dữ liệu để tránh lưu thông tin không hợp lệ vào cơ sở dữ liệu.

5. Hướng xử lý

Để xử lý chức năng phân trang, em bổ sung các state ở frontend như page, totalPage và totalRecord. Khi người dùng chọn trang khác, state page sẽ thay đổi và useEffect sẽ gọi lại API để cập nhật danh sách công việc. Ở backend, em sử dụng limit để giới hạn số lượng công việc mỗi trang và skip để bỏ qua các bản ghi của những trang trước.

Với trang chi tiết công việc, em tách riêng route /job/detail/:id để xử lý yêu cầu xem chi tiết. Backend lấy id từ URL, tìm công việc trong bảng Job, sau đó dùng companyId để lấy thêm thông tin công ty trong bảng AccountCompany. Các dữ liệu cần thiết được gom lại thành đối tượng jobDetail và trả về cho frontend.

Đối với form ứng tuyển, em sử dụng JustValidate ở frontend để kiểm tra dữ liệu trước khi gửi. Khi dữ liệu hợp lệ, em dùng FormData để gửi thông tin ứng viên kèm file CV lên backend. Ở backend, em sử dụng multer để xử lý upload file, Joi để validate dữ liệu và model CV để lưu thông tin ứng tuyển vào MongoDB.

6. Kế hoạch tuần tiếp theo

Trong tuần tiếp theo, em dự kiến tiếp tục phát triển và hoàn thiện thêm các chức năng của website tuyển dụng việc làm, tập trung vào phần hiển thị công ty và quản lý CV cho nhà tuyển dụng.

Cụ thể, em sẽ xây dựng trang danh sách công ty ở phía frontend để người dùng có thể xem các công ty đang có trên hệ thống. Đồng thời, em sẽ bổ sung chức năng phân trang cho trang danh sách công ty nhằm giúp giao diện hiển thị gọn gàng hơn khi có nhiều dữ liệu.

Bên cạnh đó, em sẽ phát triển trang chi tiết công ty, bao gồm cả phần frontend và backend. Ở phía backend, hệ thống sẽ xây dựng API lấy thông tin chi tiết của công ty theo id. Ở phía frontend, giao diện sẽ gọi API này để hiển thị các thông tin như tên công ty, logo, địa chỉ, mô hình công ty, số lượng nhân viên, thời gian làm việc và các thông tin liên quan.

Ngoài ra, em sẽ tiếp tục xây dựng chức năng quản lý CV dành cho nhà tuyển dụng. Chức năng này cho phép nhà tuyển dụng xem danh sách các CV mà ứng viên đã gửi, kiểm tra thông tin ứng viên, file CV, trạng thái đã xem và trạng thái xử lý hồ sơ. Đây là chức năng quan trọng để hoàn thiện luồng ứng tuyển trên hệ thống.

Cuối cùng, em sẽ kiểm tra lại các chức năng đã làm như tìm kiếm, phân trang, xem chi tiết công việc và gửi CV để đảm bảo hệ thống hoạt động ổn định, dữ liệu hiển thị chính xác và giao diện dễ sử dụng hơn.